A VISUAL FIELD GUIDE TO THE REPOSITORY

HealthLens-AI

Understand the app from the first Streamlit rerun to the final prediction or cited lesson: what each project area does, which files call which, and where the data and models fit.

1	7	32	4
main UI entry point	active Python helper modules	curated local knowledge entries	focused test modules

FOLLOW A FEATURE

Browser > **app.py** > **helpers** > **data + models** > **result**

A practical map of the code, data, models and call paths

Updated November 2026 | Built from the checked-in repository

Educational software, not a medical service.

Model outputs are demonstrations—not diagnoses or calibrated clinical probabilities.

How to use this guide

Start with the architecture, then jump to a feature. Each section explains the user journey and points to the exact code, data, and model files involved.

01 The project in one minute Purpose, app layers, and safety boundary	**02 Architecture & request flow** Startup, routing, reruns, and shared helpers
03 Prediction pages Symptom, liver, heart, diabetes, Parkinson, risk score	**04 Image classifiers** Malaria, pneumonia, and Alzheimer's MRI flows
05 Vitalis AI chatbot Local retrieval, citations, Groq, safety branches	**06 Health Literacy Tutor** Three-step lesson, answer checks, XP, session state
07 Python file map Responsibilities and direct call graph	**08 Data & knowledge** CSV/TSV schemas and TF-IDF retrieval
09 Models & assets Saved artifacts, visuals, samples, trust boundary	**10 Tests & supporting files** Configuration, notebook, unused archive
11 Run, test & maintain Commands, retraining, and maintenance checklist	**12 End-to-end reference** Quick call-chain lookup

Reading note: the app combines older prediction demos with newer source-grounded chatbot and tutor modules. Where the guide calls out implementation caveats, those are behaviors visible in the checked-in code, not clinical evaluations.

01 / START HERE

The project in one minute

HealthLens is a single-process Streamlit app. It brings together educational prediction demonstrations and a source-grounded chatbot and tutor. The UI and page orchestration live in one large script; reusable logic for symptom ranking, image inference, retrieval, Groq, and tutoring lives in `code/`.

The interface

Sidebar navigation chooses a page. Streamlit reruns `app.py` as users interact; page blocks show forms, images, chat, and results.

The prediction demos

Inputs go to saved classifiers or, for liver prediction, a decision tree fitted from a local CSV during the page run.

The source-grounded assistant

Questions are matched to local reviewed passages. The chatbot may ask Groq to word a response using those passages; some safety-sensitive replies are composed locally.

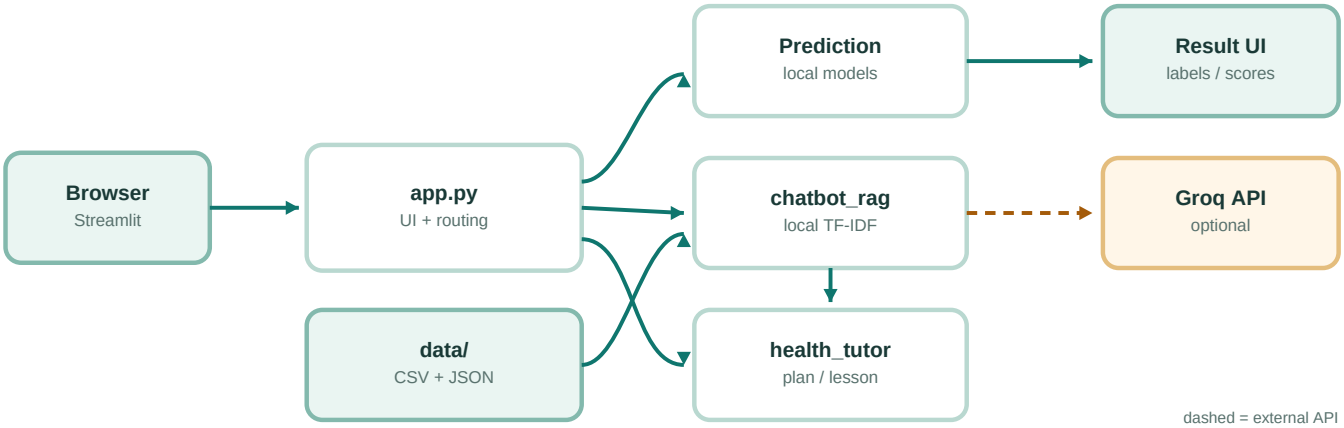
The learning tutor

A three-step plan, short lessons, and answer checks are composed from reviewed passages and tracked in the current Streamlit session.

02 / HOW THE APP STARTS

Architecture & request flow

The menu is the routing mechanism. There is no separate API server, database, or front-end bundle in the active app.



Most calls are direct Python function calls inside the Streamlit process. Groq is the only external service in the active chatbot/tutor path.

Configure. `app.py` imports packages, reads the favicon, sets Streamlit page config, and injects global CSS.

Route. The sidebar menu sets `selected`; the matching conditional page block renders.

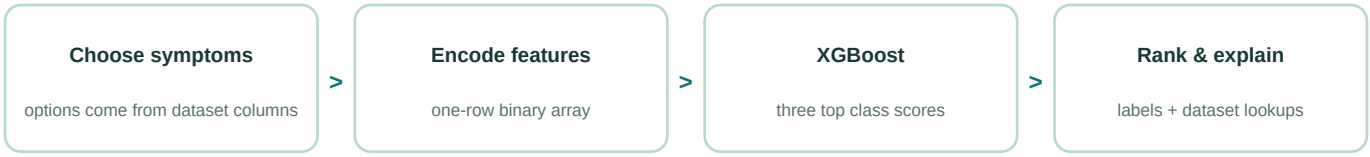
Collect. Widgets capture symptoms, measurements, images, chat text, or tutor answers.

Compute. The page calls a model/helper, retrieval module, or tutor service. Streamlit reruns the script on interactions.

Show. Results are rendered in the same page. Chat and tutor progress live in `st.session_state`, not durable storage.

Prediction pages: input to result

Symptom-based ranking



`app.py` creates `DiseaseModel`, loads `models/xgboost_model.json`, and offers `all_symptoms`. `prepare_symptoms_array()` maps chosen symptom names to a fixed-size vector using `data/clean_dataset.tsv`. `DiseaseModel.predict_top_k()` calls XGBoost `predict_proba`, then `rank_disease_probabilities()` maps model classes to the ordered disease labels. The UI shows three matches, descriptions from `symptom_Description.csv`, and optional precaution text from `symptom_precaution.csv`.

The displayed scores are ranking scores, not calibrated clinical probabilities. Empty symptom selection is stopped with a warning.

Measurement and questionnaire pages

Page	Inputs -> computation	Artifact / output
Liver Disease Prediction	Ten sliders; read liver CSV, median-fill missing ratio, train/test split, fit a new decision tree during page run, then predict.	`data/liver_dataset.csv` -> binary class.
Heart Disease Prediction	13 cardiovascular sliders -> single-row feature input -> classifier `predict`.	`models/heart_disease_model.pkl` -> binary class.
Diabetes Prediction	Eight numeric values in pregnancy, glucose, pressure, skin, insulin, BMI, pedigree, age order -> `predict`.	`models/diabetes_model.sav` -> binary class.
Parkison Prediction	22 voice/acoustic measurements, assembled in model feature order -> `predict`.	`models/parkinsons_model.sav` -> binary class.
Risk Factor Assessment	Checkboxes -> hand-coded weights (2 or 3) summed; <5 Low, <10 Medium, otherwise High.	No learned model; fixed demonstration rule.

Image classifiers

Cell image

Upload JPG/PNG -> resize to 224 x 224 -> array shape `[1,224,224,3]`, scale by 1/255 -> load `CNN_Model.h5` -> threshold first output at 0.5 -> infected / not infected message.

When it runs: model loads on upload; inference waits for Predict. Assumes three color channels.

Chest X-ray

Upload JPG/JPEG/PNG -> RGB -> resize 224 x 224 -> scale by 1/255 -> load `pneumonia_classifiers.h5` -> call `predict` -> threshold first output at 0.5.

When it runs: prediction occurs on upload. A checkbox changes the preview, not the inference.

MRI category

Upload -> convert RGB -> cached model load -> resize 244 x 244 -> MobileNetV2 preprocessing -> predicted class and score.

Model builder: `code/alzheimer.py` constructs a DenseNet201 backbone with pooling and dense layers and loads `alzheimer_model.h5` weights.

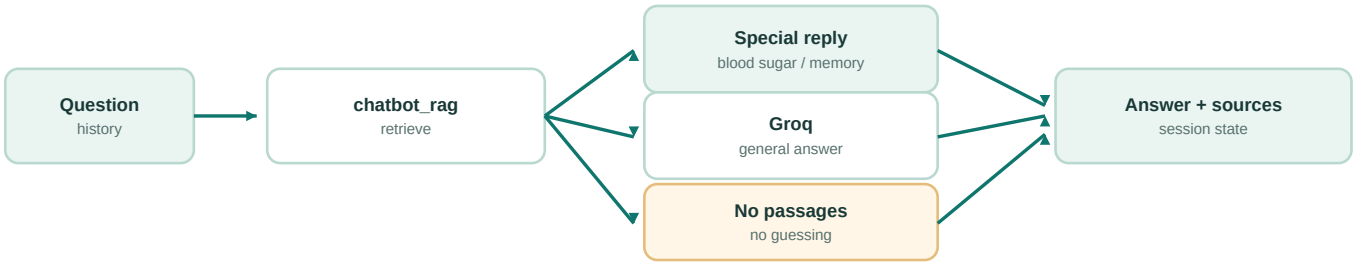
Alzheimer inference call chain

```
app.py: upload + image validation
  ■■ get_alzheimer_model()              [@st.cache_resource]
       ■■ code.alzheimer.load_alzheimer_model()
            ■■ build DenseNet201 inference graph
            ■■ load models/alzheimer_model.h5 weights
  ■■ code.alzheimer.predict_alzheimer_image(image, model)
       ■■ RGB + resize + MobileNetV2 preprocessing
       ■■ model.predict -> argmax -> four class names + score
```

The class names are Mild Demented, Moderate Demented, Non Demented, and Very MildDemented. The helper uses nearest-neighbor resizing. Confirm that this preprocessing matches how the weights were trained: the helper builds DenseNet201 but uses MobileNetV2 preprocessing.

Vitalis AI: local references, optional generation

Search happens locally over the reviewed JSON knowledge base. Groq generates language only for the general-answer branch using the retrieved passages and safety instructions.



Two targeted answer paths avoid free-form generation; unsupported questions receive an explicit "not enough reviewed information" response.

`build_retrieval_query()` adds the latest user question only when the new message looks like a follow-up.

`retrieve_passages()` loads the local index once, filters topic mismatches, and returns passages with source IDs.

High-blood-sugar action questions use `build_high_blood_sugar_guidance()`; memory-loss concern questions use `build_memory_loss_information()`.

No relevant passages means the app declines to guess. Otherwise, context and chat history are passed through `stream_groq_response()`.

The UI streams text, normalizes generated text, shows sources with links/review dates, and stores turns in `st.session_state.chat_messages`.

06 / LEARNING JOURNEY

Health Literacy Tutor: a three-step loop

`health_tutor.py` validates that a plan has exactly three non-empty title/objective pairs, maps lesson citations to local passages, requests an adaptive lesson, and checks answers as structured JSON. It never trusts model-generated source IDs for citations.

Mastered

+25 XP, streak increments, step advances, mastery count increments, and the next lesson can become slightly more challenging.

Not mastered yet

Supportive feedback, no points lost, streak resets, attempt number increments, and the next lesson re-explains the objective.

State includes topic, passages, plan, step index, attempt number, lesson, feedback, mastery status, XP, streak, and completion flag. It exists in the current browser session only. Topics: Alzheimer's disease and memory, diabetes, heart disease, liver disease, malaria, Parkinson's disease, and pneumonia.

What each Python file does

`app.py` Main Streamlit application: theme, navigation, all page UI, input validation/rendering, helper calls, session state, and error messages.

`code/__init__.py` Makes `code` importable as a Python package.

`code/DiseaseModel.py` Loads the XGBoost model, exposes symptom and disease labels, predicts/ranks labels, and reads disease descriptions and precautions.

`code/disease_ranking.py` Validates top-k and class/probability alignment, orders scores, and maps class IDs to disease names.

`code/helper.py` Maps selected symptom names to the feature-column order in the TSV and creates the model input array.

`code/alzheimer.py` Defines MRI class names/size, constructs the DenseNet201 graph, loads weights, preprocesses an image, and returns predicted label/score.

`code/chatbot_rag.py` Loads and validates JSON references, builds cached TF-IDF vectors, retrieves topic-filtered passages, and prepares contexts/citations/special-case answers.

`code/groq_client.py` Central Groq HTTP client: request construction, timeout, normal and streaming completions, error handling, and text normalization.

`code/health_tutor.py` Defines supported topics and prompt rules; retrieves tutor sources; creates and validates plans, lessons, and answer feedback.

`code/train.py` Manual symptom model-training script: preprocess raw dataset, write cleaned feature TSV, fit/evaluate XGBoost, save artifacts.

Direct call graph

```
app.py
■■ Symptom page
■  ■■ helper.prepare_symptoms_array()
■  ■■ DiseaseModel.predict_top_k()
■     ■■ disease_ranking.rank_disease_probabilities()
■■ Alzheimer page
■  ■■ alzheimer.load_alzheimer_model() / predict_alzheimer_image()
■■ Vitalis AI
■  ■■ chatbot_rag.retrieve_passages() / format...() / unique_sources()
■  ■■ groq_client.stream_groq_response()
■■ Health Literacy Tutor
   ■■ health_tutor.create_learning_plan() / generate_lesson() / evaluate_answer()
      ■■ chatbot_rag.retrieve_passages()
      ■■ groq_client.generate_groq_response()
```

Data files & knowledge retrieval

File	Contents	Used by
`data/dataset.csv`	Disease target plus sparse symptom columns.	`code/train.py`
`data/clean_dataset.tsv`	Generated multi-hot symptom matrix plus disease label; establishes inference feature/label ordering.	Training, `code/helper.py`, `DiseaseModel.py`
`data/symptom_Description.csv`	Disease label and description.	Symptom match cards.
`data/symptom_precaution.csv`	Disease label and up to four dataset-provided precaution fields.	Symptom page expander.
`data/liver_dataset.csv`	Age, sex, liver measurements, and target column.	Liver page trains a decision tree at runtime.
`data/chatbot_knowledge.json`	32 reviewed entries with ID, title, text, source name, URL, and optional review date; covers seven health topics plus HealthLens scope.	Chatbot and tutor retrieval.

How the TF-IDF retrieval works

Read and validate the JSON records; each must include ID, title, text, and source name.

Build a cached scikit-learn TF-IDF index over title + text (English stop words, unigrams and bigrams, sublinear term frequency).

Vectorize the question and rank passages by cosine similarity; apply minimum relevance, topic matching, special-query requirements, and source diversity.

Return passage text and citation/source metadata. This is local lexical retrieval-not live web search.

Models, assets & sample images

Saved models in `models/`

Artifact	Consumer	Format / role
`xgboost_model.json`	`DiseaseModel`	XGBoost model for symptom ranking.
`alzheimer_model.h5`	`code/alzheimer.py`	Weights loaded into the reconstructed DenseNet201 graph.
`CNN_Model.h5`	Malaria page	Keras image classifier.
`pneumonia_classifiers.h5`	Pneumonia page	Keras image classifier.
`heart_disease_model.pkl`	Heart page	Pickled classifier.
`diabetes_model.sav`	Diabetes page	Joblib-loaded classifier.
`parkinsons_model.sav`	Parkison page	Joblib-loaded classifier.

Trust boundary: pickle/joblib can execute code when loaded. Only use model files from trusted sources. The repository does not fully document training provenance for every saved artifact.

Visual assets in `assets/`

`healthlens.png` - favicon and sidebar logo; `home_hero.png` - landing page.

`symptoms_illustration.png` - symptom page; `heart_illustration.png`, `liver_illustration.png`, `diabetes_illustration.png`, `parkinsons_illustration.png` - measurement/voice pages.

`malaria_illustration.png`, `pneumonia_illustration.png` - image pages.

`alzheimer_illustration.svg` - embedded in the MRI page; `alzheimer_logo.svg` - additional logo artwork.

Root-level `alzheimer.jpg`, `nonAlzheimer.png`, `Pneumonia.jpeg.png`, `non-Pneumonia.jpeg`, `malaria.jpg`, and `no-malaria.jpeg` are sample images; the active pages accept uploads rather than loading them by name.

Tests, configuration & historical files

Tests

`tests/test_chatbot_rag.py` - retrieval, special routing, sources and context.
`tests/test_groq_client.py` - HTTP payloads, response parsing, streaming and failures (mocked network).
`tests/test_health_tutor.py` - topic retrieval and plan/feedback validation.
`tests/test_disease_ranking.py` - score ordering, class mapping and invalid input checks.

Config & overview

`README.md` - tutor, key setup, privacy, and medical-use scope.
`requirements.txt` - Streamlit, data/ML, TensorFlow/Keras, image and UI dependencies.
`.streamlit/config.toml` - light theme colors.
`.gitignore` - virtualenv, caches, archives, secrets.
`.vscode/settings.json` - workspace editor settings.

The Alzheimer's notebook

`augmented-alzheimer-mri-with-densenet-92-5.ipynb` contains 18 code cells for loading/splitting MRI data, image generators, DenseNet201 model training, evaluation, saving, prediction, accuracy/loss plots, and metrics. It is an experiment/training artifact-not part of normal app startup.

What is in `unused/`?

Historical/legacy material: an older duplicate `DiseaseModel.py`; compiled Python caches; old diabetes, Parkinson, hepatitis, lung cancer, and plant-disease datasets/models; a standalone Alzheimer script; old images and assets. Active code uses the root `code/`, `data/`, `assets/`, and `models/` directories. The listed unused files are retained history, not active imports in the checked-in app flow.

Dependency note

`requirements.txt` pins several packages but leaves core dependencies such as NumPy, Streamlit, and TensorFlow unpinned. TensorFlow H5, pickle/joblib, and XGBoost model compatibility can depend on library versions.

Run it, test it, change it

Local run

```
python -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
streamlit run app.py
```

Run from the repository root because several paths are relative. Configure `GROQ_API_KEY` in the environment or Streamlit secrets before using the chatbot or tutor. Never commit secrets.

Test

```
python -m unittest discover -s tests
```

The existing suite focuses on the extracted retrieval, Groq client, tutor service, and ranking functions. It does not comprehensively exercise the Streamlit pages or every saved model.

Retrain the symptom model

Review `data/dataset.csv` and the intended symptom columns.

Run `python code/train.py` from the repository root. It regenerates `data/clean_dataset.tsv` and saves the XGBoost JSON model (the script also writes a compressed joblib artifact).

Verify training feature and label order against `helper.py` and `DiseaseModel.py`.

Run tests and trace a known input through feature vector, model class, and displayed label.

Maintenance checklist

Relative paths require launching at the repository root.

Diabetes and Parkinson artifact deserialization runs at module import/rerun, even on unrelated pages.

The Liver decision tree trains inside the page script; confirm data split, feature ordering, and expected reproducibility.

Check image channel assumptions, resize/preprocessing parity, and model output meanings against training code.

Session state is temporary; the app has no user account, database, or durable tutor progress.

Make the educational/non-diagnostic boundary obvious whenever changing result language.

End-to-end: who calls whom

User action	Call chain	What comes back
Select symptoms	`app.py -> helper.py -> DiseaseModel.py -> disease_ranking.py`	Top-three label matches; data CSV description/precautions.
Submit liver inputs	`app.py -> liver CSV -> DecisionTreeClassifier.fit/predict`	Binary model label.
Heart / diabetes / Parkinson	`app.py -> saved pickle/joblib model.predict`	Binary model label.
Upload image	`app.py -> PIL/NumPy preprocessing -> Keras model.predict`	Thresholded image label; Alzheimer uses the helper and four classes.
Ask Vitalis AI	`app.py -> chatbot_rag.py -> special answer OR groq_client.py -> render_sources()`	Grounded answer, citations, and source metadata.
Use the tutor	`app.py -> health_tutor.py -> chatbot_rag.py + groq_client.py`	Validated three-step learning loop and session-local progress.
Retrain symptom model	`code/train.py + data/dataset.csv`	Clean TSV and XGBoost model artifact.

Prepared from the checked-in repository. This is an implementation guide, not a certification of model accuracy, clinical validity, dataset quality, or privacy compliance.